Urban Fork Restaurant App MVP - SRS v2

Assignment 2 | Version 2.0 | Fall 2026

Student Name: _____ | Registration No.: _____

Derived from existing A1/SRS.pdf. Preserves sections 1-7, subsections 1.1-1.3, 2.1-2.3, 7.1-7.8 and FR-01-FR-35. Replaces Section 5 and adds Section 8. Original A1 files are unchanged; SRS_v1_extracted.txt preserves the extracted original text.

1. Introduction

1.1 Purpose

This SRS defines version 2.0 of the existing Urban Fork restaurant MVP for customers and managers. It extends the A1 React Native/Expo interface, navigation, themes, contexts, reducers and reusable components with an Express REST API and MongoDB. The audience remains the instructor, student developer and prototype reviewers/testers.

1.2 Scope

Keep A1 menu browsing, categories, search, favourites, cart quantities/notes/promotions, dine-in/takeaway receipts, reservations, tracking, manager operations and themes. A2 adds Express, REST APIs, MongoDB/Atlas, Mongoose, bcrypt hashes, one-day JWT authentication, server authorization/validation and multi-client consistency. AsyncStorage stores token/public user; MongoDB owns menu, orders and reservations.

A1 local-data problem	A2 solution
Edits exist only on one phone	Shared menu API/database; customer refresh shows changes
Mock credentials and session-only signup	Hashed users and verified JWT login/registration
Signup can select manager role	Public customers only; protected manager APIs
Local bookings miss other customers	Shared availability and unique active-slot index
Timers pretend food is ready	Real manager transitions, 10-second customer polling
Client totals can be edited or stale	Server recalculates database prices
Local orders never reach kitchen	Central orders/reservations and manager APIs

Out of scope: real payments/payment gateways, push notifications, production hosting/hardening and WebSocket streaming. API requests/polling support shared state. Atlas verification and physical-device evidence are manual; no credentials or evidence are fabricated.

1.3 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification.
MVP	Minimum Viable Product.
UML	Unified Modeling Language.
Hook	React state/lifecycle function.
Context	Shared React state without prop drilling.

Term	Definition
Reducer	Pure state/action transition function.
Mock Data	A1 fixtures retained as seed provenance.
FlatList	Optimized React Native scrollable list.
AsyncStorage	Local key-value storage, now used for token/user session.
Debounce	Wait for typing to pause before search.
API	Application Programming Interface for app/server communication.
REST	Resource-based HTTP API conventions.
Endpoint	HTTP method/path identifying an operation.
JSON	JavaScript Object Notation request/response format.
MongoDB	Shared document database.
Collection	Named group of MongoDB documents.
Mongoose	Node.js MongoDB modeling, validation and queries.
Middleware	Express functions before controllers or during error handling.
JWT	Signed JSON Web Token proving session; expires after one day.
HTTP Status Code	201 created, 400 invalid, 401 unauthenticated, 403 forbidden, 404 missing, 409 conflict, 500 unexpected failure.

2. Overall Description

2.1 User Roles

Role Key Permissions & Goals

Customer / Diner

Create an account, log in, browse/search menu, manage favourites, add items to cart, apply promo codes, reserve a table, place an order, track status, view profile, switch theme and logout.

Restaurant Manager

Log in, access a manager-only dashboard, update incoming orders, accept/decline reservations, add menu items, edit prices, toggle availability, view profile, switch theme and logout.

2.2 Customer Self-Interview

As a potential Urban Fork customer, I would download the app if it helped me decide what to eat before reaching the restaurant, showed accurate prices and availability, let me reserve a table, gave me a clear bill before placing an order, and showed whether my food was pending, preparing, ready or served. I would also

value favourites, recent searches, dark mode and the ability to add notes such as “less spicy” or “no onions.” These needs directly shape the user stories and functional requirements below.

2.3 User Stories

1. As a Customer, I want to browse the full menu with prices and availability, so that I can choose food before arriving.

2. As a Customer, I want to search for dishes by name or description, so that I can find food quickly.

3. As a Customer, I want to filter the menu by category, so that I can focus on relevant dishes.

4. As a Customer, I want to save favourite dishes, so that I can quickly return to items I like.

5. As a Customer, I want to reserve a table for a selected date and time, so that I can avoid waiting.

6. As a Customer, I want to add special instructions to cart items, so that my meal matches my preference.

7. As a Customer, I want to apply a valid promo code, so that I can see the discount before ordering.

8. As a Customer, I want to see a detailed order summary, so that I understand charges, tax, discount and total.

9. As a Customer, I want to choose Dine-in or Takeaway, so that the order matches how I plan to receive it.

10. As a Customer, I want to track order progress, so that I know when the food is ready.

11. As a Restaurant Manager, I want to view and update incoming orders, so that kitchen progress is reflected in the app.

12. As a Restaurant Manager, I want to accept or decline reservations, so that table usage remains manageable.

13. As a Restaurant Manager, I want to add menu items, edit prices and toggle availability, so that the customer menu stays current.

14. As a Restaurant Manager, I want to use a role-restricted dashboard, so that customer-only screens do not appear in my workflow.

3. Functional Requirements

A1 requirement identities are preserved; authentication/data/status requirements are revised for A2.

Module	ID	Requirement (system shall)
Authentication	FR-01	Allow switching between Login and Signup modes on one screen.
Authentication	FR-02	Validate email, password of 8-72 characters including a digit, and matching password confirmation.
Authentication	FR-03	Authenticate through Express login with bcrypt verification; public registration creates only customers.
Authentication	FR-04	Route customers to customer navigation and managers to the existing dashboard.
Authentication	FR-05	Store token/user in AuthContext and AsyncStorage; restore startup session and clear both on logout.
Menu	FR-06	Load at least fifteen items in four A1 categories from GET /api/menu; seed all twenty A1 items.
Menu	FR-07	Display name, description, price, bundled image, category, availability and Daily Special state.

Module	ID	Requirement (system shall)
Menu	FR-08	Grey out unavailable items and disable Add; server also rejects unavailable ordered items.
Menu	FR-09	Keep category filtering and sorting by ascending/descending price and name.
Menu	FR-10	Keep customer favourite toggles in screen state.
Search	FR-11	Search loaded names/descriptions; API also supports case-insensitive name search combined with category.
Search	FR-12	Debounce search input by exactly 400 milliseconds.
Search	FR-13	Retain the last five distinct search terms as tappable suggestions.
Search	FR-14	Show Back to Top after scrolling more than 300 pixels.
Search	FR-15	Show friendly empty, loading, error and Retry states; pull-to-refresh retrieves server menu.
Cart	FR-16	Allow available menu items to be added to the cart.
Cart	FR-17	Allow increment/decrement and remove an item when quantity reaches zero.
Cart	FR-18	Store special instructions per item; enforce maximum length on server.
Cart	FR-19	Preview URBAN10, DINE15 and FEAST20 locally; server validates codes and discounts.
Cart	FR-20	Display the live cart-item-count badge.
Reservation	FR-21	Offer hourly slots from 12:00 to 22:00.
Reservation	FR-22	Disable slots when the server reports no suitable unoccupied table, including other customer bookings.
Reservation	FR-23	Keep date, party-size 1-12, 03XX-XXXXXXX phone and one-hour lead-time validation; enforce on server in Asia/Karachi.
Reservation	FR-24	Show confirmation before POST; allow owners to cancel active reservations through an additional API.
Orders	FR-25	Allow Dine-in or Takeaway; server validates table existence or pickup selection.
Orders	FR-26	Preview subtotal, 5% service charge, promo discount, 15% sales tax and total; server recalculates from current database prices.
Orders	FR-27	POST order with ObjectId, price/name snapshots, Pending status and timestamps; clear cart only on success.
Orders	FR-28	Replace simulated progression with GET /api/orders/my immediately and every exactly 10 seconds; clean up interval.
Orders	FR-29	Keep progress indicator and elapsed time; statuses change only after manager API actions.
Manager Dashboard	FR-30	Display manager navigation only to managers; enforce authorization independently on server.
Manager Dashboard	FR-31	Allow Pending -> Preparing/Cancelled, Preparing -> Ready and Ready -> Served only.
Manager Dashboard	FR-32	Accept/Decline only pending reservations; declined/cancelled slots become available again.

Module	ID	Requirement (system shall)
Manager Dashboard	FR-33	Add menu items, edit prices and toggle availability with protected APIs; refetch after successful mutations.
Manager Dashboard	FR-34	Persist changes centrally; customer refresh loads latest prices; checkout always uses database prices.
Manager Dashboard	FR-35	Store menu/orders/reservations in MongoDB instead of AsyncStorage; preserve local cart, favourites, recent searches and theme.

4. Non-Functional Requirements

Area	Requirement
Usability	Preserve readable labels, validation, loading/error/Retry and confirmations.
Performance	Keep FlatList, React.memo and memoized filtering/totals; use real fetch state; poll every 10 seconds and clean up.
Responsiveness	Preserve safe areas and common portrait phone layouts.
Maintainability	Keep existing frontend structure; add src/api and separate server models/routes/controllers/config/middleware/tests.
Data Handling	MongoDB stores shared records; AsyncStorage stores public user/token. Idempotent seed preserves existing records.
Reliability	Validate client/server input; retain cart on failure; reject illegal transitions and conflicts; ignore stale responses.
Accessibility	Descriptive labels/touch targets; status text complements colour.
Compatibility	Retain Expo SDK 57; run dependency checks and web/Android exports where supported.
Security	bcrypt, 1-day JWT, server roles, whitelisted fields, hidden hashes, ignored .env.

5. Database Design

All five named collections use unique _id: ObjectId and createdAt/updatedAt: Date timestamps. Unique constraints use MongoDB indexes. References use ObjectIds; old A1 ids are only seed provenance (legacyId/table code).

5.1 Users

Field/type	Required	Constraints/default
name: String	Yes	trimmed; length 2-100
email: String	Yes	unique; lowercase; trimmed; email pattern
password: String	Yes	bcrypt hash; select:false; removed by JSON transform
role: String	Yes	customer/manager enum; default customer

One User has many Orders/Reservations. Public registration rejects manager roles; seed provisions managers.

5.2 MenuItems

Field/type	Required	Constraints/default
name: String	Yes	nonempty, trimmed, max 100
description: String	Yes	nonempty, trimmed, max 500
price: Number	Yes	min 0; finite JSON number
category: String	Yes	Starters/Mains/Desserts/Drinks enum
imageKey: String	No	default truffle-parmesan-fries; bundled A1 image
legacyId: String	No	unique sparse index; seed idempotence
isSpecial: Boolean	No	default false
isAvailable: Boolean	No	default true

MenuItems are referenced by many order lines; historical orders retain price/name snapshots after edits/deletion.

5.3 Tables

Field/type	Required	Constraints/default
code: String	Yes	unique; A1 T01?T07
name: String	Yes	trimmed
seats: Number	Yes	integer; min 1; max 12
area: String	Yes	restaurant area

One Table has many Reservations on different slots and can be referenced by many dine-in Orders.

5.4 Reservations

Field/type	Required	Constraints/default
customer: ObjectId	Yes	ref User
table: ObjectId	Yes	ref Table; server checks existence
tableName: String	Yes	database snapshot
date: String	Yes	YYYY-MM-DD; real date and one-hour lead time
time: String	Yes	hourly 12:00-22:00; Asia/Karachi
partySize: Number	Yes	integer; min 1; max 12; <= table seats
contactName: String	Yes	trimmed; max 100
phone: String	Yes	03XX-XXXXXXX pattern
status: String	Yes	Pending/Accepted/Declined/Cancelled; default Pending
active: Boolean	No	default true; server controlled

Unique compound partial index (table,date,time) where active:true blocks simultaneous conflicts. Pending/Accepted occupy; Declined/Cancelled release. Each Reservation belongs to one User and Table. Timestamps enabled.

5.5 Orders

Field/type	Required	Constraints/default
customer: ObjectId	Yes	ref User; from authenticated user
items: Array	Yes	nonempty; max 100 distinct lines at API
items.menuItem: ObjectId	Yes	ref MenuItem; existence/availability checked
items.name: String	Yes	database snapshot
items.price: Number	Yes	database snapshot; min 0
items.quantity: Number	Yes	integer; min 1; max 99
items.note: String	No	default empty; max 500
type: String	Yes	Dine-in/Takeaway enum
table: ObjectId	No	ref Table; default null; required by controller for dine-in
tableName: String	No	default empty; snapshot
pickupTime: String	No	default empty; validated A1 choices for takeaway
promoCode: String	No	default null; URBAN10/DINE15/FEAST20 allowlist
subtotal/serviceCharge/promoDiscount/salesTax/total : Number	Yes	min 0; server computed
status: String	Yes	Pending/Preparing/Ready/Served/Cancelled; default Pending

Order belongs to one User and references one or more MenuItems. Server ignores client total/price/owner/status. Subtotal uses database prices, service charge is 5%, promotion is server validated, sales tax is 15% of discounted taxable amount; final PKR total rounds to whole rupees, matching A1. Timestamps enabled. Legal transitions: Pending -> Preparing or Cancelled; Preparing -> Ready; Ready -> Served. Atomic compare-and-set detects concurrent manager edits. Served/Cancelled are terminal.

Cart, favourites, categories, recent search and promo definitions remain frontend state/constants.

6. UML Diagrams

Preserve original A1/UML diagrams and A1/UML_Diagrams.pdf:

1. Use Case Diagram: existing Customer/Manager workflows; API additionally enforces roles.

2. Class Diagram: existing UI/domain classes; Section 5 adds database relationships.

3. Sequence Diagram: updated A2 order flow in diagrams/order-sequence.mmd and .svg.

4. State Machine Diagram: A2 uses manager events only; A1 timer events no longer change statuses.

5. Component Diagram: A2 adds central API client, Express and Mongoose.

6. Deployment Diagram: diagrams/deployment.mmd and .svg shows Phone -> Laptop/Express -> MongoDB Atlas.

Both A2 diagrams are also in diagrams/A2_Diagrams.pdf.

7. MVP Frontend Development (React Native)

7.1 Login and Signup Screen

Keep original form, validation, password visibility and ActivityIndicator. Use /auth/login and /auth/register; public role choice offers Customer only. AuthContext persists token/user, restores through /auth/me, clears session on logout. Navigation entry/exit is unchanged.

7.2 Menu Browsing Screen

Keep chips, FlatList, bundled images, specials, availability, favourites, sorting/Add, refresh/loading/error/Retry. useApi(refreshMenu) retrieves /menu; no mock or AsyncStorage menu fallback. Manager-created dishes retain the house image fallback.

7.3 Search and Scroll Controls

Keep exact 400ms debounce, five recent searches, input/list/render refs, Back to Top and memoized name/description search. API category/name filters can combine separately.

7.4 Profile and Theme Screen

Keep name/email/role, global theme switch and logout confirmation; user details originate on server. Cart resets on session change. Existing AppNavigator keeps role-based navigation.

7.5 Cart Screen

Keep CartContext/useReducer, quantity/note/remove actions, badge, promotions and receipt preview. Server total governs checkout; failed submissions retain cart.

7.6 Order Summary Screen

Keep receipt, type/pickup selectors. Fetch database tables. POST item IDs, quantities, notes, type, table/pickup and promotion; trust server response/total and clear cart only on success. Display errors without false confirmation.

7.7 Table Reservation Screen

Keep useReservation, frontend validation and confirmation/cancellation UI. Query availability for all hourly slots, including other customer occupancy without exposing identities. POST, GET /my and owner cancellation use server. Periodic/pull refresh retrieves current decisions.

7.8 Order Tracking and Manager Dashboard

Keep tracking cards, progress and elapsed time. OrdersScreen useEffect fetches immediately, setInterval(...,10000), cleanup clearInterval; fake advancement removed. ManagerDashboard uses useApi, polling/pull refresh and protected orders/reservations/menu APIs. Mutations refetch server menu; Accept/Decline and legal order transitions update server. Existing OrdersContext/reducer displays authoritative responses.

8. API Endpoints

Base: http://<LAPTOP_LAN_IP>:5000/api. IDs are MongoDB ObjectIds. Protected routes require Authorization: Bearer <token>, valid for exactly 86400 seconds. JSON error bodies contain message. All protected routes may return 401 missing/invalid/expired credentials or 403 wrong role. Unknown route: 404 {message: "Route not found"}; unexpected error: sanitized 500.

Method	Path	Access	Main statuses	Body/result
GET	/api/health	Public	200	status/time

Method	Path	Access	Main statuses	Body/result
POST	/api/auth/register	Public	201/400/409	name,email,password; public customer only; token/user
POST	/api/auth/login	Public	200/400/401	email,password; JWT/user
GET	/api/auth/me	Authenticated	200/401	public user
GET	/api/menu	Public	200/400	optional category/search; array
GET	/api/menu/:id	Public	200/400/404	menu record
POST	/api/menu	Manager	201/400/401/403	name,description,price,category; optional flags/imageKey
PUT	/api/menu/:id	Manager	200/400/404	allowed fields with validators
DELETE	/api/menu/:id	Manager	200/400/404	deletion message
GET	/api/tables	Public	200	database table records
GET	/api/tables/availability	Public	200/400	date,time,partySize; unoccupied tables
POST	/api/orders	Customer	201/400	items[{menuItem,quantity,note}],type,tableId or pickupTime,promoCode
GET	/api/orders/my	Customer	200	own orders only
GET	/api/orders	Manager	200	all orders
PATCH	/api/orders/:id/status	Manager	200/400/404/409	status; legal transition and atomic update
POST	/api/reservations	Customer	201/400/409	tableId,date,time,partySize,contactName,phone
GET	/api/reservations/my	Customer	200	own reservations
GET	/api/reservations	Manager	200	all reservations
PATCH	/api/reservations/:id	Manager	200/400/404/409	Accepted/Declined; pending only
PATCH	/api/reservations/:id/cancel	Customer owner	200/400/404/409	cancel active own booking

8.1 Example order request

```
{"items":[{"menuItem":"<MENU_OBJECT_ID>","quantity":2,"note":"Less
spicy"}],"type":"Takeaway","pickupTime":"In 20 minutes","promoCode":"URBAN10"}
```

Server returns _id, customer, price/name snapshots, computed receipt, Pending status and timestamps. Frontend adds display aliases id/timestamp. Do not rely on client total.

8.2 Example reservation request

```
{"tableId":"<TABLE_OBJECT_ID>","date":"<FUTURE_YYYY-MM-
DD>","time":"19:00","partySize":2,"contactName":"Ayaan","phone":"0300-1234567"}
```

Active-slot uniqueness returns 409 for conflicts. Accept preserves occupancy; Decline/owner Cancel releases it.

Appendix A: Preserved A1 Implementation Notes

A.1 Filtering dependency note (Q4)

If a filtering effect has an empty dependency array, it runs only after the first mount. Changes to the selected category or loaded menu items would not trigger the effect again, so the displayed list could become stale. When state is used for derived filtering, the changing inputs must be dependencies. In the optimized version, derived filter/search/sort output should be calculated with useMemo instead of stored as separate state.

A.2 Context vs prop drilling (Q6)

Authentication and theme data are required by screens at different levels of the navigation tree. Context lets screens read the current user or theme directly without passing the same props through intermediate components that do not need them. It gives login, logout and theme switching one shared source of truth. Focused hooks such as useAuth and useTheme keep screen code clear. Role-based navigation can react to AuthContext immediately. A drawback is that consumers may re-render when the provided context value changes, so context values should be designed carefully.

A.3 useReducer vs useState for Cart (Q7)

The cart has many related transitions: add, remove, increment, decrement, update note, clear cart, apply promo and remove promo. useReducer centralizes these transitions in one pure function and makes changes predictable through explicit actions. This structure is easier to test and maintain when several fields change together. useState is still appropriate for simple isolated values such as a search query, selected category or modal visibility.

A.4 Reducer Test Cases (Q7)

Action Initial State Expected State

ADD_ITEM items=[] Selected item exists with quantity=1.

INCREMENT item quantity=1 Same item quantity=2.

DECREMENT item quantity=1 Item is removed when quantity reaches zero.

UPDATE_NOTE note="" Item note equals the entered instruction.

APPLY_PROMO promoCode="", discountPercent=0 Valid promo stored and discount updated.

REMOVE_PROMO discountPercent>0 Promo code and discount reset.

CLEAR_CART items contain products items=[] and promo fields reset.

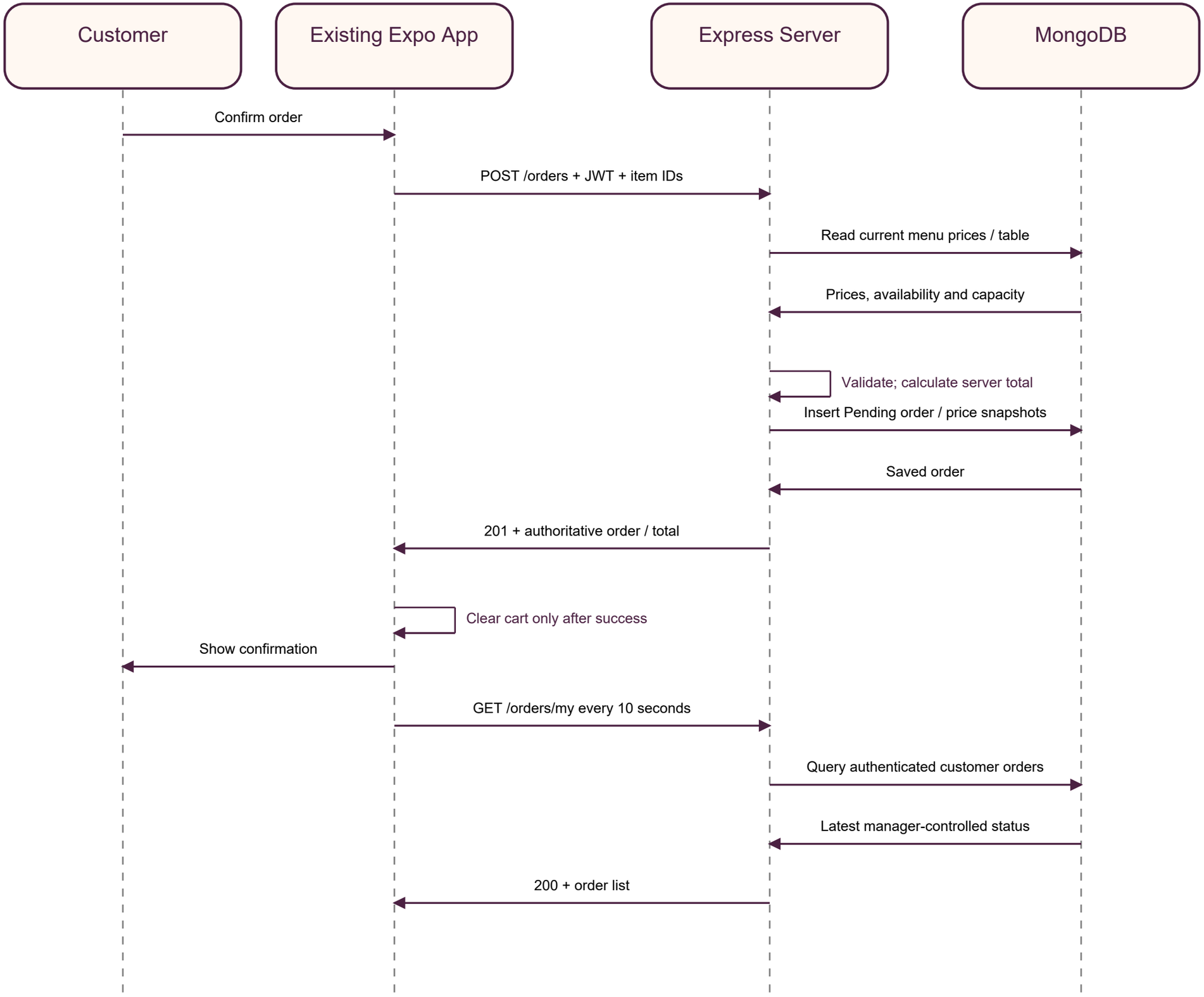
A.5 When not to use useMemo/useCallback (Q8)

useMemo and useCallback should not be added to every value or function automatically. They have bookkeeping cost and can make code harder to read. They are most useful when a calculation is expensive, a derived list is recomputed often, or stable references prevent unnecessary rendering of memoized children. For small calculations and ordinary event handlers without a measurable performance problem, normal values and functions are simpler. Optimization should be used for a clear reason rather than by default.

Revision validation and evidence

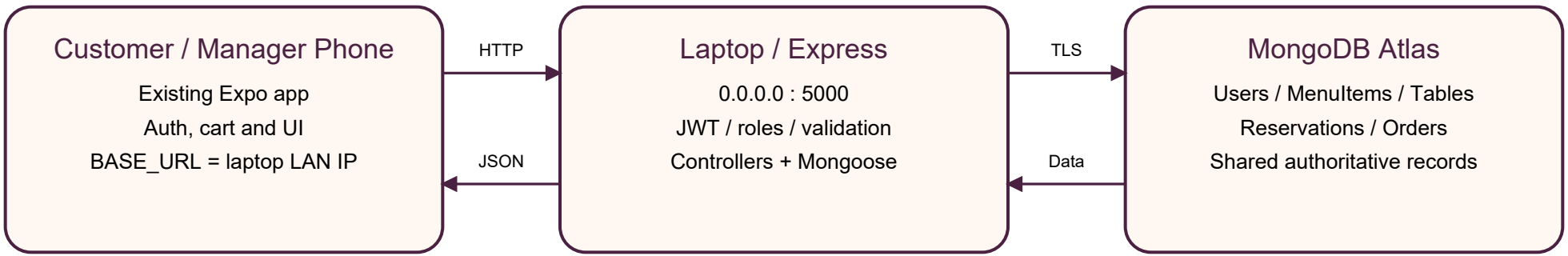
Tests use disposable MongoDB, not Atlas. Atlas verification, genuine screenshots, physical two-client price test and <=4-minute recording remain manual. See the evidence instructions and TEST_RESULTS.md. This SRS does not claim submission evidence is complete.

Customer places an order - Assignment 2



Failed POST retains cart. Statuses never advance from local demo timers.

Deployment - Phone, Laptop and MongoDB Atlas



Phone -> Laptop: JSON REST requests + Bearer token; same Wi-Fi / permitted firewall port.

Laptop -> Atlas: private MONGO_URI stays in ignored server/.env; never bundled in the app.

Orders poll every exactly 10 seconds. Atlas / physical-device verification remains manual.